

Draw It or Lose It
CS 230 Project Software Design Template
Version 1.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 2
Requirements 2
Design Constraints 2
System Architecture View 3
Domain Model 3
Evaluation 4
Recommendations 7

Document Revision History

Version	Date	Author	Comments
1.0	10/15/2026	Ethan Motquin	Initial software design document

Executive Summary

The Gaming Room currently has Draw It or Lose It available as an Android application and wants to expand the game into a web-based application that can support multiple platforms. The application allows teams to compete by identifying images that are gradually rendered as clues. The primary software requirements are to support one or more teams per game, allow multiple players on each team, maintain unique game and team names, and ensure that only one instance of the game service exists in memory at a time.

The proposed solution uses an object-oriented design with an Entity base class and specialized Game, Team, and Player classes. The GameService class manages games and uses the Singleton design pattern to ensure that only one GameService instance exists. The Iterator pattern is used when searching existing games, teams, and players so that duplicate names are not created. Unique identifiers are also assigned to games, teams, and players. This design provides a structured foundation that can be expanded into a web-based application capable of supporting multiple client platforms.

Requirements

The Gaming Room requires Draw It or Lose It to support one or more teams within a game. Each team must be able to contain multiple players. Game and team names must be unique so that users can determine whether a name is already being used. Games, teams, and players must also have unique identifiers. In addition, only one instance of GameService should exist in memory at any given time. The application should use object-oriented design principles and software design patterns to efficiently manage these requirements.

Design Constraints

The primary design constraint is that Draw It or Lose It must operate as a web-based application in a distributed environment. Users may access the application from different types of computers and mobile devices, so the software must be designed to communicate reliably across different platforms. The application should avoid depending on a specific client operating system whenever possible.

Another constraint is maintaining consistent game information across clients. Games, teams, and players require unique identifiers and names so that the application can correctly identify objects and prevent duplicate names. The GameService Singleton also creates a constraint because only one instance of the service should exist in memory. This provides a single point for managing game information.

The application must also account for network connectivity because a web-based game depends on communication between clients and the server. Temporary connectivity problems or server outages could prevent users from accessing game information. The design should therefore separate the client interface from the core game logic so that the application can support multiple platforms while maintaining consistent business rules.

Security is another important constraint because the application will communicate across a network and may handle user information. Communication between clients and the server should be protected, and access to game information should be controlled appropriately.

System Architecture View

The system can use a client-server, multi-tier architecture. Players use a responsive web interface in their browsers, the application tier manages game logic and services, and a data tier can provide persistent storage when needed. This separation allows the client interface, game logic, and data storage to be maintained independently while communicating through defined interfaces.

Domain Model

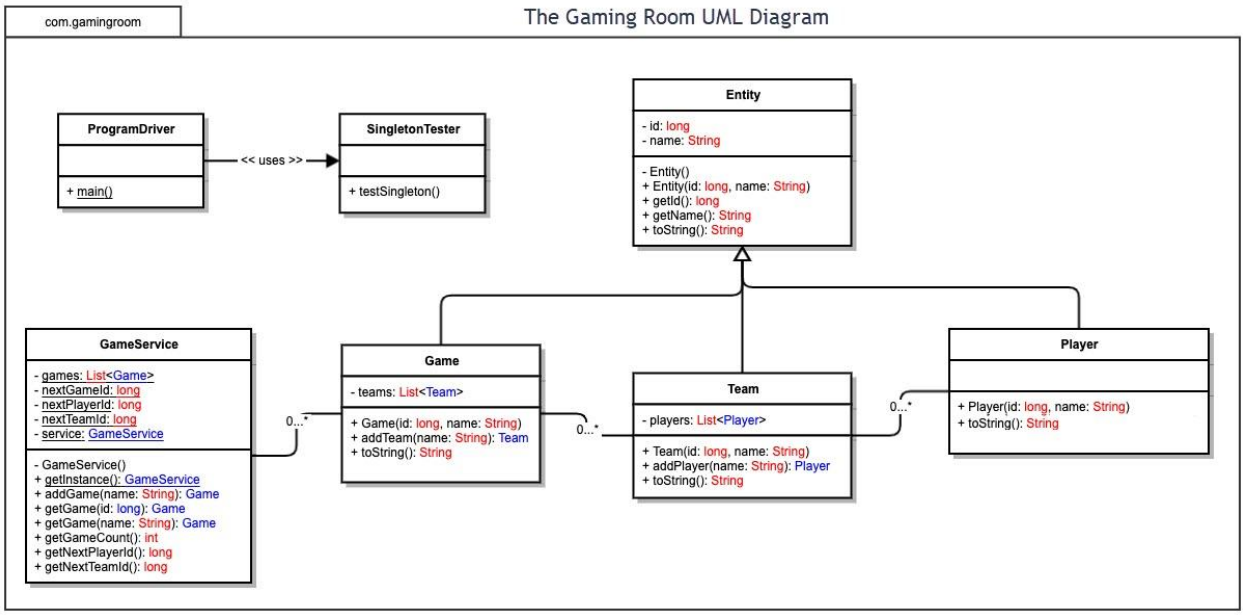
The UML class diagram represents the main objects used by the Draw It or Lose It application. Entity is the base class and contains the common id and name attributes along with the getId(), getName(), and toString() methods. Game, Team, and Player inherit from Entity, allowing these classes to reuse the common attributes and behaviors instead of defining them separately.

GameService manages the collection of games and contains the methods required to add and retrieve games. It also contains the identifiers used to assign unique IDs to games, teams, and players. The GameService class uses the Singleton design pattern through a private constructor and a shared instance returned by getInstance(). This ensures that only one GameService object exists in memory.

A Game can contain zero or more teams, represented by the List<Team> collection. A Team can contain zero or more players, represented by the List<Player> collection. GameService maintains a collection of zero or more games. These relationships allow the application to organize its information in a hierarchy where the service manages games, games manage teams, and teams manage players.

The diagram demonstrates several object-oriented programming principles. Inheritance is demonstrated because Game, Team, and Player derive from Entity. Encapsulation is

demonstrated by keeping object data within the classes and accessing it through methods such as getId() and getName(). The design also demonstrates abstraction because Entity provides the common characteristics shared by the different types of game objects. The Singleton and Iterator design patterns further support the software requirements by controlling the GameService instance and allowing the application to search collections for existing games, teams, and players.



Evaluation

The evaluation compares Mac, Linux, Windows, and mobile devices for server-side development, client-side compatibility, and development tools. The main recommendation is to host the web application on Linux while using a common browser-based client so that players on desktop and mobile operating systems can access the game.

Development Requirements	Mac	Linux	Windows	Mobile Devices

Server Side	A Mac can be used to develop and test a web based application because it provides a full desktop operating system and supports common development environments. A major advantage is that developers can use modern web browsers and Java development tools. A limitation is that development teams may need access to Apple hardware and macOS specific tools when testing software intended specifically for Apple devices.	Linux is well suited for hosting web based applications because it provides a stable server environment and supports common programming languages, development tools, databases, and web servers. Linux can also be used with cloud-based infrastructure. A potential weakness is that developers may need additional Linux-specific knowledge when configuring servers, networking, permissions, and security.	Windows provides a familiar desktop environment and supports Java development tools, web browsers, databases, and IDEs such as Eclipse. It can be useful for developers who already have Windows systems. A consideration is that server deployments may require additional configuration and administration depending on the technologies selected for the application.	Mobile devices are important client platforms because Draw It or Lose It is being expanded beyond its original Android application. A web-based design can allow users to access the game through a mobile browser without requiring a separate version of the complete game logic for every mobile operating system. The application should use a responsive interface that works with different screen sizes and input methods. Network connectivity and device performance should also be considered.

Client Side	Supporting Mac clients requires testing the web application in commonly used browsers on macOS. The development team should verify that the user interface and game functionality work correctly across supported browsers. The primary costs are development time, browser testing, and personnel with web development experience. Because the application is web based, separate native Mac game software should not be necessary.	Supporting Linux clients requires browser compatibility testing and consideration of differences between Linux distributions and browsers. The application should rely on standard web technologies rather than operating system-specific functionality. Development costs can be reduced by using a common web interface, although developers may need additional time to test different Linux configurations.	Windows clients can access the game through a supported web browser. The development team should test the application on commonly used Windows browsers to ensure consistent functionality. Development time is primarily associated with compatibility testing rather than creating a separate Windows version of the game.	Mobile clients require additional consideration because screen sizes, hardware capabilities, and input methods vary between devices. The web interface should use responsive design so that the game can be used on smartphones and tablets. Testing should include different screen sizes and mobile browsers. Supporting a common web application can reduce the need to develop separate native applications for every mobile platform.

Development Tools	Development for Mac can use Java, HTML, CSS, and JavaScript along with an integrated development environment such as Eclipse or another Java IDE. Web browsers can be used to test the client interface. Version control and debugging tools can also be used to manage and test the application.	Linux development can use Java and standard web technologies such as HTML, CSS, and JavaScript. Eclipse and other development environments can be used for Java development. Linux also provides access to common server, database, networking, and security tools that can support the application.	Windows development can use Java with Eclipse or another Java IDE. HTML, CSS, and JavaScript can be used to develop the web based client interface. Web browsers, debugging tools, database software, and version-control tools can be used to develop and test the application.	Mobile clients can use standard web technologies including HTML, CSS, and JavaScript through a mobile web browser. Responsive web design techniques can allow the same application to support different mobile screen sizes. Testing should be performed on both Android and iOS devices to identify differences in browser behavior and device capabilities.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**: A Linux-based server platform is recommended for the web-based version of Draw It or Lose It. Linux provides a stable environment for hosting web applications and supports common web servers, programming languages, databases, and security technologies. Linux can be deployed on physical servers or cloud infrastructure and scaled as the number of players and simultaneous games increases. The client application can remain browser based so players using Windows, macOS, Linux, Android, and iOS can access the same game without requiring a separate implementation of the complete game logic for each operating system.

2. **Operating Systems Architectures**: Linux uses a layered architecture. The hardware provides the physical computing resources, while the Linux kernel manages processes, memory, devices, and other core operating system functions. System libraries provide interfaces that applications use to access operating system services, and user-space applications run above these components. Linux supports multitasking, multiple user accounts, process isolation, and virtual memory, which are useful for running a game service alongside web-server and database processes.

The recommended application architecture is a multi-tier client-server design. The client tier contains a responsive web interface accessed through browsers on desktop computers and mobile devices. The application tier contains the game logic and services that manage games, teams, players, and sessions. A data tier stores information that must persist after the application stops. Separating these responsibilities allows each tier to be maintained and scaled more independently.

Windows, macOS, Linux, Android, and iOS differ in their kernels, system libraries, application frameworks, and supported hardware. Some devices use x86-64 processors while many modern mobile devices and some desktop computers use ARM processors. A native application may therefore require platform-specific development and testing. A browser-based interface built with HTML, CSS, and JavaScript reduces this dependency. The Linux server can expose a consistent API to all supported clients, while GameService manages game operations on the server. The server should remain the authoritative source of game state so all clients follow the same rules.

3. **Storage Management**: The Linux server should use a file system such as ext4 for operating system files, application files, configuration files, and logs. A file system organizes data into files and directories, tracks storage space, and supports permissions that control which users and processes can read or modify files. Application files, database files, and game image assets should be organized so that they can be backed up and maintained safely.

   A relational database such as PostgreSQL is recommended for persistent game information. Tables could represent games, teams, players, and user accounts, with unique identifiers connecting related records. Primary keys, foreign keys, and uniqueness constraints can help preserve relationships and prevent duplicate values where uniqueness is required. Database transactions can keep related changes consistent. Clients should communicate with the application server rather than connecting directly to the database.

   Image files used as game clues should be kept in a dedicated asset directory or object storage service. The database can store references to those assets instead of storing large image files directly in records. Data that must survive a restart should be saved in persistent storage rather than existing only in RAM. The Gaming Room should monitor available storage, restrict access to sensitive files, back up important data regularly, and test restoration procedures so information can be recovered after a failure.

4. **Memory Management**: Linux manages physical RAM by allocating memory to running processes and provides virtual memory so each process has its own address space. Virtual memory helps isolate processes and allows the operating system to manage memory under changing workloads. Disk-backed swap space can help when RAM is under pressure, but it is much slower than physical memory and should not be treated as a substitute for sufficient RAM.

   Draw It or Lose It should keep only the information needed for active games in application memory, such as current sessions, team membership, and frequently used game state. Information that must remain available after the application stops should be stored in the database. Caching frequently requested images or data can improve response times, but caches should have limits so they do not consume excessive memory.

The GameService Singleton supports the design requirement that one shared GameService instance manages the game collection within a running application instance. It does not, by itself, guarantee that only one service instance can exist across multiple server processes or machines. If the application later uses multiple server instances, shared state and coordination will be needed. Lists and other collections can hold active game objects, and resources should be released when games end or players disconnect. In Java, garbage collection can reclaim memory occupied by objects that are no longer reachable. Monitoring memory usage and cleaning up inactive sessions can help prevent memory exhaustion as demand grows.

5. **Distributed Systems and Networks**: The game should use a distributed client-server architecture. Players access the web interface through their devices, and the server validates requests, updates the authoritative game state, and sends updates back to connected clients. HTTPS should protect communication. A REST API can handle operations such as signing in, joining a game, or retrieving team information. WebSockets can be used when active games need real-time updates to be delivered to multiple players.

   The system depends on the client device, network connection, application service, database, and image storage working together. A network interruption can prevent a player from receiving updates. The client should detect a lost connection, display a clear message, and attempt to reconnect. After reconnection, it should retrieve the latest valid game state. Requests that may be repeated during reconnection should be designed to avoid performing the same action more than once.

   Server health checks, logs, monitoring, backups, and tested recovery procedures can help the team identify and respond to failures. As usage increases, a load balancer and multiple application-server instances could spread requests across servers. This change would require appropriate session management and shared persistent data so users receive consistent information regardless of which server handles a request. The system should explain when a service is unavailable rather than leaving the interface unresponsive.

6. **Security**: Security should be applied at the operating system, application, database, and network levels. Linux user accounts and file permissions should follow the principle of least privilege, giving each service only the permissions it needs. Administrative access should be restricted to authorized personnel, and a firewall should allow only the network services required for operation. Linux, the database, and application dependencies should be kept updated with security patches.

   HTTPS using properly configured TLS should encrypt information transmitted between players and the server. Authentication should verify a user's identity, while authorization should determine which games and actions that user can access. Session tokens should be protected and invalidated when no longer needed. The server should validate input rather than trusting values sent by clients. Database queries should use parameterized statements to reduce the risk of SQL injection, and database accounts should have only the permissions required for their functions.

Passwords should never be stored as plain text; they should be protected using a suitable password-hashing algorithm with unique salts. Sensitive data should not be collected or retained unless it is needed. Stored information and backups should be protected with appropriate access controls and encryption where warranted. Security logs and monitoring can help identify suspicious activity, but logs should not expose passwords or session tokens. Regular security testing, incident-response procedures, and tested backups would help The Gaming Room protect user information and recover from an outage or security incident.

Conclusion

A Linux-based server combined with a browser-based, multi-tier client-server architecture is an appropriate foundation for expanding Draw It or Lose It to multiple operating systems. A relational database and organized file storage can preserve game information, while Linux memory management, application cleanup, and monitoring can support reliable operation. Standard network protocols, reconnection handling, and layered security controls would help maintain consistent gameplay and protect user information as the game grows.

References

Ubuntu. (n.d.). Ubuntu Server. https://ubuntu.com/server

PostgreSQL Global Development Group. (n.d.). PostgreSQL documentation. https://www.postgresql.org/docs/

National Institute of Standards and Technology. (n.d.). Cybersecurity resources. https://www.nist.gov/cybersecurity